MIT 8103 Advanced Database Systems

Bank / Fintech Platform · Olumuyiwa Olaifa · Student ID 301918345

Case study

My case study focuses on a Bank / Fintech Platform designed around three main entities: customers, accounts and payments. The database uses separate tables for each entity to keep customer and account information organised while maintaining accurate payment records.

The platform supports common banking operations, including creating customer accounts, recording payments and retrieving transaction histories. It also allows operations staff to check payment statuses, review transactions within a specified date range and summarise settled payments by account and currency.

To examine query performance, I prepared a separate benchmark using 50,000 synthetic payment records. I compared the performance of an initiated_at date-range query before and after creating a B-tree index. The comparison considered execution plans, returned records and query execution times.

Portfolio 1: Database Design and Modelling

I structured the database around three relational tables: Customer, Account and Payment. Each table has a primary key, while foreign keys maintain the relationships between customers, their accounts and associated payments. I also included CHECK and UNIQUE constraints to enforce the defined business rules.

The design applies normalisation principles across 1NF, 2NF and 3NF, using sample records to demonstrate how the data is organised at each stage. Particular attention was given to payment references: where a reference is provided, it must be unique within its account.

I tested the constraints using 25 invalid insert statements. PostgreSQL rejected all 25 invalid statements, leaving the existing valid records unchanged.

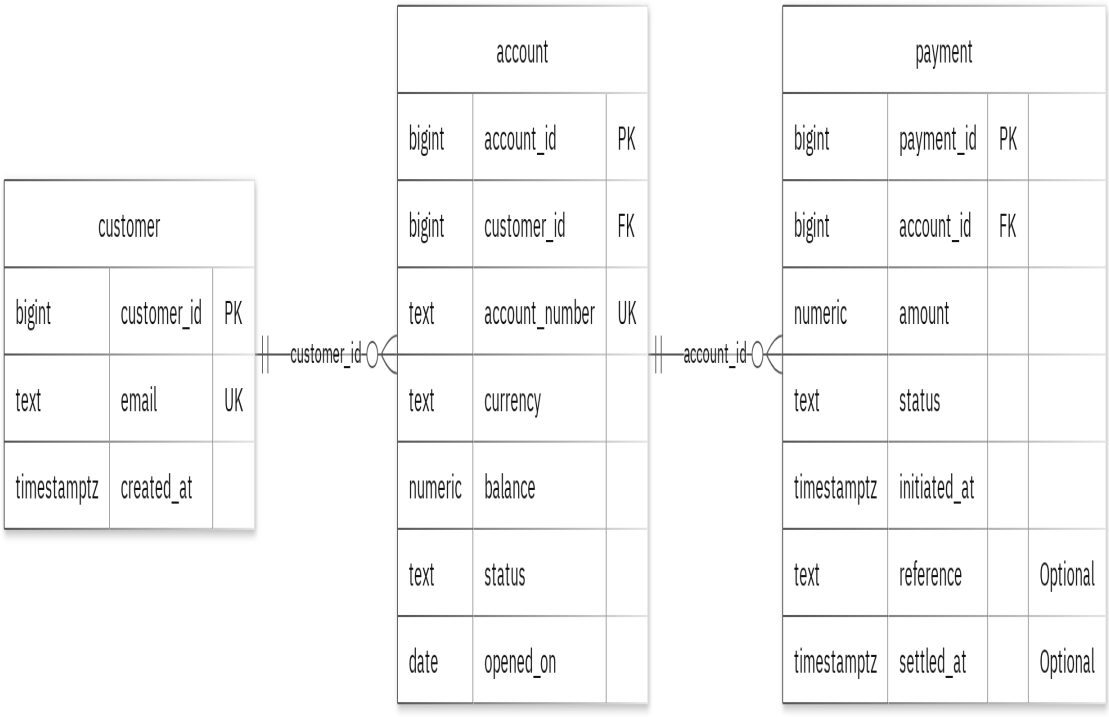


Figure 1. Tables, keys and relationships. PK: primary key; FK: foreign key; UK: unique value.

Portfolio 2: Query Processing and Optimisation

I developed and executed three SQL queries against the sample database to retrieve payment information and examine the relationships between customers, accounts and transactions.

The join query returned a record: the account ACCT-2001 and the associated customer email: customer1@example.test.

For the performance test, I used a separate dataset of 50,000 synthetic payment records. The date-range query returned the same records before and after indexing, but its median execution time decreased from 2.231 ms to 0.234 ms after the B-tree index was introduced.

The execution plan also showed that PostgreSQL used the new index. These results demonstrate the improvement observed for the query and test environment; they should not be treated as a general performance benchmark for every database workload.

Portfolio 3: Transactions and Concurrency

I used a temporary account counter to examine transaction behaviour under COMMIT, ROLLBACK and READ COMMITTED isolation.

The exercise showed that changes made in an uncommitted transaction were not visible to the second session. When the first session rolled back its transaction, the counter returned to its previously committed value.

I also tested concurrent updates using two database sessions without row locking; both sessions read the same initial value and their updates resulted in a final counter value of 2. When I introduced the SELECT FOR UPDATE; Session B had to wait for Session A to complete its transaction. Both increments were then preserved, giving a final value of 3.

The counter was created specifically for this test, so the original customer, account and payment records remained unchanged.

Portfolio 4: NoSQL and Advanced Data Models

I extended the case study with a MongoDB collection named payment_events to represent payment-related event documents.

The exercise involved: inserting two sample events, running four queries to retrieve and examine the stored documents, and testing the collection’s validation rules by inserting an invalid date type and a duplicate event ID; the samples were rejected.

The document structure uses embedded snapshots to retain relevant account and customer details alongside each event. This makes the information available within the event document, while the details field accommodates differences between event types.

The events are fictional and serve as test data. The relational database remains responsible for enforcing the original customer, account and payment relationships.

Repository

https://github.com/oolaifa8345/Olaifa*Olumuyiwa*301918345

The repository contains the SQL and JavaScript scripts, database explanations, execution plans, query results and screenshots for the practical exercises.

Portfolio 5: Distributed and Cloud Database Exercise

For the distributed database exercise, I proposed a MongoDB replica set consisting of three members deployed across separate failure zones.

The design discussion considers how primary elections work, the role of majority write acknowledgement and the effect of secondary replication lag. I also examined the trade-offs involved in sharding the payment_events collection.

The proposed arrangement uses replication to support database availability, while backups provide a separate means of recovering data. Since PostgreSQL also forms part of the case study, it would require its own availability and recovery arrangements.

This portfolio was completed as a design discussion, as permitted by the exercise. I did not deploy a live replica set or carry out an actual failover test.

Individual reflection

Working through the exercises helped me understand how database design decisions affect how banking information is stored, retrieved, and updated.

Keeping customer, account and payment information in separate tables reduces unnecessary duplication and makes it easier to maintain relationships between records. I also saw how constraints protect the database against incorrect entries, particularly where payment references must remain unique within an account.

The concurrency exercise was particularly useful when two sessions read the same counter value before updating it; one increment could be lost. Introducing row-level locking resolved this issue by making the second session wait until the first transaction was completed.

The indexing exercise also reinforced the need to examine execution plans when assessing query performance. Although both queries returned identical results, the indexed version completed faster in my test. This showed me that obtaining the correct result and retrieving it efficiently are separate considerations in database development.

Overall, the exercises gave me practical experience with relational modelling, query optimisation, transaction management, document databases and distributed database design within a banking context.

AI-use declaration

I used GitHub Copilot and the portfolio generator to assist with preparing the database model, scripts and accompanying explanations. The generator executed the PostgreSQL and MongoDB exercises, checked their outputs and captured the result images.

I reviewed the generated materials and take responsibility for the accuracy of the submitted work, including the database implementations, test results and explanations.

References

- PostgreSQL: constraints
- PostgreSQL: EXPLAIN
- PostgreSQL: isolation
- PostgreSQL: row locks

- MongoDB: data modelling
- MongoDB: replication
- MongoDB: sharding
- Mermaid: ER diagrams